

The Opus Audio Codec: Theory and Implementation

Jack Geraghty

Contents

1	Introduction	2
1.1	The Reference Implementation	2
1.2	Target Audience and Prerequisites	2
1.3	Scope and Organization	3
1.4	Notation Conventions	3
2	Architecture Overview	3
2.1	Two Codecs, One Bitstream	3
2.2	The Layered Stack	4
2.3	Decoding a Packet: An Overview	4
2.4	Sample Rates and the 48 kHz Reference	5
2.5	From Quantities to Symbols	6
3	The Range Coder	6
3.1	Why Entropy Coding	6
3.2	The Integer Range Coder	7
3.2.1	Decoding a symbol	7
3.2.2	The renormalization loop	8
3.2.3	Encoding a symbol	8
3.3	Carry Propagation	8
3.4	The ICDF Representation	9
3.5	Binary Symbols and Raw Bits	10
3.6	Bit Budgeting: tell() and tell_frac()	11
3.7	The Final-Range Oracle	12
4	Python API Reference	12
4.1	Introduction	12

1 Introduction

This manual provides a comprehensive overview of the algorithms, optimizations, and design decisions that make up the Opus audio codec [rfc6716]. The focus is on understanding both the theoretical foundations and the practical implementations of the techniques Opus combines: the information theory of its entropy coder, the linear prediction of its speech path, the transform coding of its music path, and the fixed-point and SIMD machinery that lets all of it run in real time.

The material emphasizes the *why* and *how* of each component: *why* certain design choices matter, and *how* they are implemented efficiently. It covers both the mathematics and the practical computational considerations that make these methods work in production systems.

Opus is the IETF’s general-purpose, royalty-free audio codec, standardized in 2012 and updated by RFC 8251 [rfc8251]. It is the default audio codec of WebRTC and is widely deployed in streaming and conferencing. Its defining characteristic is that it is not one codec but two, combined behind a single bitstream. A linear-predictive speech coder (*SILK*) and a transform-domain music coder (*CELT*, the Constrained-Energy Lapped Transform) share one entropy coder, one packet format, and one container. The encoder selects, per frame, between SILK, CELT, or both together (the *hybrid* mode). The combination spans a range no single technique covers well: 6 kb/s narrowband speech to 510 kb/s fullband stereo music, frame sizes from 2.5 ms to 60 ms, and sample rates from 8 kHz to 48 kHz, all decoded by one receiver that is not told in advance which mode each packet uses.

1.1 The Reference Implementation

Throughout, the implementation under discussion is `opus_native`, a pure-Rust, dependency-free realization of RFC 6716. Every code listing in this manual is drawn from that source. Three of its design choices shape the discussion:

- **Bit-exact by construction.** The bitstream is defined by the range coder (Section 3), not by any one encoder’s choices. The implementation’s entropy arithmetic follows the RFC byte-for-byte, and the decoder is validated against the official test vectors on the *final-range oracle*, a per-packet checksum of the entropy-coder state that must agree with the reference to the bit.
- **Safe by default.** The crate denies `unsafe` everywhere except a handful of explicitly annotated SIMD kernels (??). The signal-processing math is ordinary safe Rust; the only machine intrinsics sit in encoder hot loops where a faster, approximate result is provably harmless (??).
- **Decode-first.** The decoder is the normative half of the specification and the half the conformance vectors exercise. The encoder is free to make any choice the decoder can follow; this asymmetry recurs throughout the manual and is the key to several optimizations.

1.2 Target Audience and Prerequisites

This material assumes familiarity with:

- Discrete signals, sampling, and the discrete Fourier transform
- Linear algebra (vectors, matrices, inner products, projection)
- Basic probability and the idea of entropy / information content
- Programming and algorithm analysis; the listings are in Rust, but no Rust expertise is required to follow them

No prior knowledge of speech coding, transform coding, arithmetic coding, or psychoacoustics is assumed; each is introduced where it is first needed. A reader who has worked through a companion treatment of FFT-based spectral analysis will find the transform-coding sections (CELT) immediately familiar, and the linear-prediction sections (SILK) a complementary story told in the time domain.

1.3 Scope and Organization

The manual is organized bottom-up, from the bit to the file, mirroring the layering of the codec itself:

1. **Architecture Overview** (Section 2) — the two codecs, three modes, the shared range coder, and the sample-rate model.
2. **The Range Coder** (Section 3) — entropy coding, arithmetic/range coding, the ICDF representation, raw bits, `tell()`, and the final-range oracle.
3. **Packet Framing** (??) — the TOC byte, the 32 configurations, frame packing codes 0–3, padding, and validity rules.
4. **CELT: Transform Coding** (??) — the MDCT, the Vorbis window and TDAC, pyramid vector quantization, CWRS enumeration, band splitting, energy coding, bit allocation, and the encoder’s analysis.
5. **SILK: Linear-Predictive Coding** (??) — linear prediction, LSF/NLSF quantization, long-term (pitch) prediction, the noise-shaping quantizer, gains, the shell coder, and resampling.
6. **The Opus Layer** (??) — mode dispatch, the SILK/CELT crossover, redundancy at transitions, packet-loss concealment, in-band FEC, and discontinuous transmission.
7. **Ogg Encapsulation** (??) — pages, the Ogg CRC, lacing, packet reassembly, `OpusHead/OpusTags`, and granule timing.
8. **Multistream and Surround** (??) — coupled and uncoupled streams, channel mapping families.
9. **Computational Optimizations** (??) — SIMD dot products, the SIMD PVQ search, the MDCT pre-rotation, the FFT backend seam, table-driven CWRS, and buffer reuse.
10. **Numerical Considerations and Conformance** (??) — fixed-point versus float, bit-exactness, the oracle, and the conformance criterion.

1.4 Notation Conventions

Throughout this manual:

- $x[n]$ denotes a time-domain sample at index n ; $X[k]$ a frequency-domain coefficient at bin k
- f_s denotes a sample rate in Hz; Opus’s internal reference rate is always 48 kHz
- N denotes a transform length or vector dimension; K a pulse count
- LM (“log multiplier”) denotes $\log_2$ of the frame-size multiplier: $\text{LM} = 0, 1, 2, 3$ for 2.5, 5, 10, 20 ms CELT frames
- The range coder’s state is the pair (val, rng)
- Qn denotes a fixed-point value scaled by 2^n : a Q15 value v represents the real number $v/2^{15}$
- $\text{sgn}(x)$ is the sign of x ; $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ are floor and ceiling
- Section references like “§4.3.4” without further qualification refer to RFC 6716 [**rfc6716**]

2 Architecture Overview

This section describes the codec as a whole: its components, the path a packet takes through them, and the design decisions that the rest of the manual elaborates. Later sections treat each layer in detail.

2.1 Two Codecs, One Bitstream

Opus is built on the observation that audio divides into two regimes that reward opposite coding strategies. The codec implements both and selects between them per frame.

Speech versus Music: Two Coding Strategies

Speech is produced by a physical system (the vocal folds exciting the vocal tract) that is well modeled as a slowly varying *linear filter* driven by a sparse *excitation*. Coding speech efficiently means coding that model: estimate the filter, estimate the pitch, and transmit only the residual that the model fails to predict. This is **linear-predictive coding** (LPC), and it is the approach SILK takes, in the *time domain*, at internal rates of 8, 12, or 16 kHz.

Music is arbitrary: dense, polyphonic, and percussive, with energy spread across the full 20 kHz audible band and no simple source model. Coding music efficiently means working in the *frequency domain*, where the ear's own frequency analysis allows bits to be spent where they are audible and withheld where masking hides them. This is **transform coding**, and it is the approach CELT takes, via the Modified Discrete Cosine Transform (MDCT) at 48 kHz.

Neither technique performs both tasks well: LPC degrades on polyphonic material, and transform coding spends bits inefficiently on the strong predictable structure of speech. Opus therefore retains both and switches between them per frame; in the overlap region (wideband speech with a musical background) it runs them together as the **hybrid** mode.

This yields the three **modes**, selected by the packet header (??) and dispatched by the decoder (??):

Mode	Coder(s)	Typical use
SILK-only	SILK	low-bitrate speech, 8–16 kHz audio band
Hybrid	SILK (low) + CELT (high)	wideband speech + full bandwidth
CELT-only	CELT	music, low delay, high bitrate

In hybrid mode the split is by frequency: SILK codes the signal up to roughly 8 kHz (it runs at a 16 kHz internal rate), and CELT codes the band above, starting at MDCT band 17 (??). Both coders write into the same range coder, so there is no boundary marker between them in the byte stream: the CELT data is simply a continuation of the same range-coded message that began with the SILK data.

2.2 The Layered Stack

Table 1 lists the layers as the implementation modularizes them, bottom to top. The lower layers (range coder, packet, Ogg) are pure framing and entropy: exact integer arithmetic, `no_std`-compatible, with no audio math. The middle layers (SILK, CELT) perform the signal processing. The top layer (the Opus decoder and encoder) provides the glue: mode dispatch, resampling, and the cross-frame state used to handle mode transitions.

Layer	Specification	Responsibility
ogg	RFC 3533 / 7845	file container, pages, timing
multistream	RFC 7845 §5.1.1	surround: N coupled/mono streams
decoder/encoder	RFC 6716 §4/§5	mode dispatch, transitions, PLC/FEC/DTX
silk	RFC 6716 §4.2	linear-predictive speech coder
celt	RFC 6716 §4.3	MDCT transform coder
packet	RFC 6716 §3	TOC byte, frame packing
range	RFC 6716 §4.1/§5.1	entropy coder

Table 1: The Opus layer stack, bottom to top.

2.3 Decoding a Packet: An Overview

To decode one Opus packet, the decoder performs the following steps:

1. Reads the first byte, the **TOC** (Table of Contents), which names the mode, audio bandwidth, frame size, and channel count (??).
2. Splits the remaining bytes into 1–48 *frames* according to the TOC’s frame-packing code.
3. For each frame, constructs a range decoder over its bytes and, depending on the mode, runs the SILK decoder, the CELT decoder, or both into a shared range decoder.
4. Resamples the decoder’s internal output (always referenced to 48 kHz) to the caller’s requested rate, and applies any cross-frame smoothing for mode transitions and redundancy (??).

The top-level dispatch is a compact `match` on the parsed TOC:

```
pub fn decode_packet(&mut self, data: &[u8]) -> Result<Vec<f32>,
PacketError> {
    if data.len() <= 1 {
        // Empty or TOC-only packet: conceal one frame (DTX / loss).
        return Ok(self.decode_lost(self.last_frame_size / self.ds));
    }
    let packet = Packet::parse(data)?;
    let toc = packet.toc();
    let mut out = Vec::new();
    for frame in packet.frames() {
        let pcm = self.decode_frame(
            frame, toc.mode(), toc.bandwidth(),
            toc.frame_size().samples_per_channel_48k(),
        );
        out.extend_from_slice(&pcm);
    }
    Ok(out)
}
```

2.4 Sample Rates and the 48 kHz Reference

The relationship between sample rates in Opus is a common source of confusion. The model is stated once here and referred to throughout.

The 48 kHz Reference Clock

Opus has three distinct notions of sample rate:

- The **audio bandwidth** (narrowband, mediumband, wideband, super-wideband, fullband) is a *property of the content*: how much of the spectrum carries signal. It is named in the TOC and caps the highest coded frequency (4, 6, 8, 12, or 20 kHz).
- The **internal rate** is the rate a coder actually runs at. SILK runs at 8, 12, or 16 kHz; CELT always runs at 48 kHz.
- The **API rate** is the rate the caller asks for (8, 12, 16, 24, or 48 kHz), at which PCM is delivered.

Crucially, **all timing is expressed in 48 kHz samples** regardless of the other two. A 20 ms frame is always 960 samples “at 48 kHz”; a 2.5 ms frame is 120. Granule positions in the Ogg container (??) count 48 kHz samples. The decoder resamples from the internal rate up to 48 kHz (CELT is already there), then *down* to the API rate by an integer factor $ds = 48000/f_s$. This single reference clock is what lets SILK frames, CELT frames, and the container agree on “when” a sample occurs even though they compute it at different rates.

2.5 From Quantities to Symbols

Every numeric quantity Opus transmits (a gain, an LSF index, a pulse vector, an energy delta) becomes a symbol in the range coder. The encoder and decoder share static probability models (**ICDF** tables, Section 3) for the structured fields and fall back to **raw bits** for the near-uniform ones. This arrangement has an important consequence:

Important Note

The bitstream is defined by the *sequence of range-coder symbols*, not by the floating-point intermediate values the encoder computed to choose them. Two encoders that pick the same symbols produce identical bytes, even if one used SIMD and **rsqrt** approximations and the other used scalar double-precision. This is why the decoder must be bit-exact (it is normative) but the encoder need only be *valid* (any choice the decoder can follow is legal). Most of ?? exploits exactly this freedom.

3 The Range Coder

Every bit Opus transmits passes through the range coder. Because the bitstream is defined entirely by the sequence of range-coder symbols, and because bit-exact conformance is verified through the coder's internal state (Section 3.7), it is the natural starting point and is treated here first and in detail.

3.1 Why Entropy Coding

Suppose a symbol s occurs with probability $p(s)$. Information theory says the *ideal* cost of transmitting it is its **self-information**, $-\log_2 p(s)$ bits [shannon1948mathematical, cover2006elements]. A common symbol (p near 1) should cost a fraction of a bit; a rare one, many bits. Over a long message drawn from a known distribution, the average cost approaches the **entropy** $H = -\sum_s p(s) \log_2 p(s)$, and no lossless code can do better.

A fixed-length code cannot reach this bound: it must spend a whole number of bits per symbol, wasting the fractional part. **Arithmetic coding**, and its integer-friendly cousin **range coding** [martin1979range, witten1987arithmetic], do reach it, by abandoning the idea that each symbol maps to its own bit pattern. Instead, the *entire message* is encoded as a single number in $[0, 1)$, and each symbol narrows the interval that number must lie in.

Range Coding by Interval Subdivision

Begin with the interval $[0, 1)$. To encode a symbol s whose cumulative distribution carves out the sub-range $[fl/ft, fh/ft)$ (low and high cumulative frequencies out of a total ft), *replace* the current interval with that fraction of itself:

$$[\ell, \ell + r) \mapsto \left[\ell + r \frac{fl}{ft}, \ell + r \frac{fh}{ft} \right). \quad (1)$$

The interval shrinks by a factor of $(fh - fl)/ft = p(s)$ at each step, so after the whole message its width is $\prod_i p(s_i) = 2^{-\sum_i -\log_2 p(s_i)}$. Transmitting *any* number inside the final interval takes about $-\log_2(\text{width}) = \sum_i -\log_2 p(s_i)$ bits, the entropy. The decoder, knowing the same model, replays the subdivisions: it sees which sub-range the received number falls in, emits that symbol, rescales, and repeats.

The practical obstacle is that the interval bounds need ever more precision as they shrink, which is impossible with finite-width integers. The fix, due to Pasco and refined by Martin, is **renormalization**: as soon as the interval is narrow enough that its top bits are settled, *emit those bits* and shift them out, restoring headroom. Opus renormalizes a byte at a time.

3.2 The Integer Range Coder

Opus’s range coder (§4.1 for the decoder, §5.1 for the encoder) works in 32-bit integers. Its state is two words:

- **rng**: the current interval width, kept in $(2^{23}, 2^{31}]$.
- **val**: on the encoder, the interval’s low end; on the decoder, the offset of the received number from the top of the interval.

The governing constants name the renormalization threshold and the byte width:

```
const SYM_BITS:  u32 = 8;           // emit/consume one byte at a time
const SYM_MAX:   u32 = 255;         // 2^8 - 1
const CODE_BITS: u32 = 32;          // width of the (val, rng) state
const CODE_TOP:  u32 = 1 << 31;     // 2^31: initial encoder range
const CODE_BOT:  u32 = 1 << 23;     // 2^23: renormalize when rng
    falls to here
const CODE_SHIFT: u32 = 23;         // CODE_BITS - SYM_BITS - 1; carry
    bit position

#[inline]
const fn ilog(x: u32) -> u32 { u32::BITS - x.leading_zeros() } // 1 +
    floor(log2 x)
```

The lower bound $rng > 2^{23}$ guarantees that the division rng/ft in Equation (1) keeps enough significant bits for the largest total frequency Opus uses ($ft \leq 2^{16}$). Note `ilog` is just the hardware count-leading-zeros instruction; it appears everywhere the coder needs $\lfloor \log_2 \rfloor$.

3.2.1 Decoding a symbol

Decoding is two steps: `decode` reports *where* the received value sits in $[0, ft)$, the caller maps that to a symbol, then `update` shrinks the state to that symbol’s sub-range and renormalizes.

```
pub fn decode(&mut self, ft: u32) -> u32 {
    // Locate val within [0, ft): the symbol s satisfies fl <= decode()
    < fh.
    ft - (self.val / (self.rng / ft) + 1).min(ft)
}

pub fn update(&mut self, fl: u32, fh: u32, ft: u32) {
    let s = self.rng / ft;
    self.val -= s * (ft - fh);           // shift past the symbols
    above this one
    self.rng = if fl > 0 { s * (fh - fl) } // interior symbol: exact
    sub-width
    else { self.rng - s * (ft - fh) }; // first symbol
    keeps remainder
    self.normalize();
}
```

Two details are easy to miss and important to get right. First, the first symbol ($fl = 0$) is given the *remainder* of the range ($rng - s(ft - fh)$), not $s \cdot fh$; this is how the coder accounts for the truncation in $s = \lfloor rng/ft \rfloor$ and stays reversible. Second, `decode` caps its result at `ft` via `.min(ft)` to defend against a corrupt stream pushing the quotient out of range.

3.2.2 The renormalization loop

When `rng` drops to 2^{23} or below, the decoder shifts in a fresh byte:

```
fn normalize(&mut self) {
    while self.rng <= CODE_BOT {
        self.nbits_total += SYM_BITS;
        self.rng <= SYM_BITS;
        // Reassemble the next 8-bit symbol from the leftover bit of
        // the
        // previous byte (high) and the top 7 bits of the new byte (low
        // ).
        let byte = self.read_byte();
        let sym = (self.leftover_bit << 7) | (byte >> 1);
        self.leftover_bit = byte & 1;
        self.val = ((self.val << SYM_BITS) + (SYM_MAX - sym)) & 0
            x7FFF_FFFF;
    }
}
```

The one-bit shimmy (`leftover_bit`) is a peculiarity of Opus's coder: the decoder works with a 7-bits-from-this-byte, 1-bit-carried representation so that the encoder's carry propagation (below) lands on byte boundaries. The `& 0x7FFF_FFFF` keeps `val` within 31 bits.

3.2.3 Encoding a symbol

The encoder mirrors the decoder exactly; it must, or the two will disagree and the stream will be undecodable:

```
pub fn encode(&mut self, fl: u32, fh: u32, ft: u32) {
    let r = self.rng / ft;
    if fl > 0 {
        self.val += self.rng - r * (ft - fl);
        self.rng = r * (fh - fl);
    } else {
        self.rng -= r * (ft - fh);
    }
    self.normalize();
}
```

The encoder's `normalize` emits the top byte of `val` instead of reading one, but is otherwise the same loop, with one complication that deserves its own treatment.

3.3 Carry Propagation

When the encoder adds to `val` (the `fl > 0` branch above), the addition can overflow into bits it has *already emitted*. A byte written out as `0xFF` might need to become `0x00` with a carry rippling into the byte before it. The encoder cannot un-write bytes, so it does not write them eagerly. Instead it buffers:


```

fn carry_out(&mut self, c: u32) {
    if c == SYM_MAX {
        self.ext += 1; // a run of 0xFF: defer, might
        all carry
    } else {
        let carry = (c >> SYM_BITS) as u8;
        if let Some(rem) = self.rem.take() {
            self.write_byte(rem + carry); // resolve the buffered
            byte
        }
        if self.ext > 0 {
            // Flush the deferred 0xFF run, +carry turns them into 0x00
            .
            let sym = (SYM_MAX + u32::from(carry)) as u8;
            for _ in 0..self.ext { self.write_byte(sym); }
            self.ext = 0;
        }
        self.rem = Some((c & SYM_MAX) as u8);
    }
}

```

Why a 0xFF Run Must Be Held

A carry into a 0xFF byte turns it into 0x00 and *propagates further left*. A carry into any other byte simply increments it and stops. So the encoder can safely emit a non-0xFF byte only once it knows whether a carry is coming, which it learns from the *next* byte. It therefore keeps one pending byte in `rem` and counts a run of 0xFFs in `ext`. When a non-0xFF byte arrives with carry c : the pending byte becomes $rem + c$, and the held 0xFFs become either 0xFF (no carry) or 0x00 (carry). This buffering is invisible to the decoder; it simply reads the bytes that finally emerge. The mechanism is what makes the encoder/decoder *symmetric to the bit*, which is the property the conformance oracle (Section 3.7) checks.

3.4 The ICDF Representation

Structured fields are coded against static probability tables. Opus stores them not as a PDF or a forward CDF but as an *inverse cumulative distribution function* (ICDF): for a model with total 2^{ftb} , the table holds $icdf[k] = 2^{ftb} - fh[k]$, the cumulative *tail* above each symbol, decreasing to a terminating zero.

Why Inverse CDFs

Two reasons. First, the decode primitive in Equation (1) naturally produces “how far down from the top is `val`”, which is exactly an upper-tail comparison, so an ICDF lets the decoder walk the table with one multiply and one compare per step and no running subtraction. Second, the tables are tiny (a handful of bytes), and storing the tail lets the common “most probable symbol first” ordering terminate early. The forward arithmetic (fl, fh) is recovered implicitly from adjacent ICDF entries.

```

pub fn decode_icdf(&mut self, icdf: &[u8], ftb: u32) -> usize {
    let d = self.val;
    let r = self.rng >> ftb;           // ft is a power of two: shift,
    don't divide
    let mut k = 0;
    let mut t = self.rng;
    let mut s = r * u32::from(icdf[0]);
    while d < s {                       // walk down the tail until val
        sits in [s, t)
        k += 1;
        t = s;
        s = r * u32::from(icdf[k]);
    }
    self.val = d - s;                   // fold update() into the same
    step
    self.rng = t - s;
    self.normalize();
    k
}

```

Because $ft = 2^{ftb}$ is a power of two, the division rng/ft becomes a shift, a small but pervasive saving, since ICDF decoding is the hottest path in SILK and in CELT's side information.

3.5 Binary Symbols and Raw Bits

Two specializations round out the coder.

Binary symbols with a power-of-two probability. Many flags have a probability of the form 2^{-logp} . These skip the table entirely:

```

pub fn decode_bit_logp(&mut self, logp: u32) -> bool {
    let s = self.rng >> logp;           // sub-range for the "1" outcome
    let bit = self.val < s;
    if !bit { self.val -= s; }
    self.rng = if bit { s } else { self.rng - s };
    self.normalize();
    bit
}

```

Raw bits. Some quantities are effectively uniform (a sign, the low bits of a pulse split), and modeling them as probabilities wastes effort. Opus codes these as *raw bits*, written at the *opposite end* of the same buffer from the arithmetic-coded data.

The Two-Ended Buffer

A range-coded packet has *two* write heads in one byte array. The arithmetic coder fills from the front, most-significant first, byte by byte as it renormalizes. Raw bits fill from the *back*, least-significant first. The two heads grow toward each other; the packet is well-formed as long as they do not cross. This costs nothing: raw bits need no probability model, so they need not interleave with the arithmetic stream, and it lets the finalizer (Section 3.6) pack the last arithmetic byte and the last raw byte into the same physical byte when they happen to fit. The decoder reads raw bits backwards from the end of the packet through a small 32-bit window, refilling a byte at a time.

```

pub fn decode_raw_bits(&mut self, bits: u32) -> u32 {
    if self.nend_bits < bits { // refill the
        window from the end
        loop {
            self.end_window |= self.read_byte_from_end() << self.
                nend_bits;
            self.nend_bits += SYM_BITS;
            if self.nend_bits > WINDOW_SIZE - SYM_BITS { break; }
        }
    }
    let ret = self.end_window & ((1 << bits) - 1); // LSB-first
    self.end_window >>= bits;
    self.nend_bits -= bits;
    ret
}

```

Uniform integers combine the two: a value in $[0, ft)$ with ft not a power of two is split into an arithmetic-coded high part (8 bits of model) and a raw-bit low part.

```

pub fn encode_uint(&mut self, t: u32, ft: u32) {
    let ftb = ilog(ft - 1);
    if ftb <= 8 {
        self.encode(t, t + 1, ft); // small: pure
        arithmetic
    } else {
        let ft_hi = ((ft - 1) >> (ftb - 8)) + 1;
        self.encode(t >> (ftb - 8), (t >> (ftb - 8)) + 1, ft_hi);
        self.encode_raw_bits(t & ((1 << (ftb - 8)) - 1), ftb - 8); //
        low bits raw
    }
}

```

3.6 Bit Budgeting: tell() and tell_frac()

Opus's bit allocation (??) needs to know, mid-frame, exactly how many bits have been spent so far, and it needs that figure to *agree* between encoder and decoder, since both run the same allocation arithmetic from the same running total. The coder exposes this through `tell` and its eighth-of-a-bit refinement `tell_frac`.

```

pub fn tell(&self) -> u32 { self.nbits_total - ilog(self.rng) }

pub(crate) fn tell_frac(nbits_total: u32, rng: u32) -> u32 {
    let mut lg = ilog(rng);
    let mut r = rng >> (lg - 16); // Q15 fractional part of log2(
    rng)
    for _ in 0..3 { // 3 iterations -> 1/8-bit
        resolution
        r = (r * r) >> 15;
        let bit = r >> 16;
        lg = 2 * lg + bit;
        r >>= bit;
    }
    nbits_total * 8 - lg
}

```

Bits Used: Total Shifted In Minus Range Headroom

The intuition: `nbits_total` counts every bit shifted into the coder. But the coder has not *committed* all of them: the current range `rng` still spans $\log_2(rng)$ bits of undecided value. So the bits actually consumed are `nbits_total` $- \log_2(rng)$. `tell()` uses the integer $\lfloor \log_2 \rfloor$ (a conservative whole-bit count); `tell_frac()` refines $\log_2(rng)$ to 3 fractional bits by the Newton-style squaring loop above, giving the 1/8-bit resolution Opus’s allocator works in. The two readouts agree on encoder and decoder because they are computed from the same $(nbits_total, rng)$.

3.7 The Final-Range Oracle

The range coder provides a single, powerful correctness check for any Opus implementation, described below.

rng-Agreement and the Final Range

After encoding (or decoding) the same sequence of symbols, the encoder and decoder must hold *identical* `rng` values, and, transitively, identical `val` and identical output bytes. RFC 6716 elevates this to a conformance tool: `OPUS_GET_FINAL_RANGE` returns the coder’s `rng` at the end of each packet, and a correct decoder’s final range must equal the reference encoder’s, *exactly*, for every packet.

This is a strong check. The entire decode (thousands of symbols across SILK and CELT, every table lookup, every renormalization) collapses into a single 32-bit number that is wrong if *any* step diverged. The `opus_native` conformance suite enforces it on all twelve official vectors: a passing decode is not merely close to the reference, it is arithmetically identical through the entropy coder. The PCM may differ by a rounding bit in the final float MDCT; the *range* may not differ at all.

In hybrid and redundancy cases (??) the packet carries more than one range-coded segment; the implementation XORs their final ranges together to form the single reported value, matching the reference’s bookkeeping.

4 Python API Reference

4.1 Introduction

Python bindings for `opus_native` are planned but not yet released. This section is reserved for the complete Python reference — installation, the encoder and decoder objects, streaming and file (Ogg Opus) helpers, and worked examples that apply the concepts from the preceding sections in practice.

Important Note

This chapter is a placeholder. The bindings (a Py03 extension module mirroring the Rust `OpusEncoder/OpusDecoder` surface, with NumPy buffer-protocol interop for PCM) are under construction. Until they ship, use the Rust API documented in ?? directly, or call the C reference `libopus` through an existing binding. The remainder of this section is intentionally left blank and will be filled in alongside the first binding release.

Installation and Imports

Decoding

Encoding

Streaming and Files

Complete API Reference